

---

# Introduction to Neural Networks

## BSAD 8310: Business Forecasting — Lecture 7

---

Department of Economics

University of Nebraska at Omaha

August 24, 2026

# Lecture Outline

---

- 1 From Trees to Neural Networks
- 2 Feedforward Networks
- 3 Training: Loss and Backpropagation
- 4 PyTorch Implementation
- 5 Key Takeaways

---

## From Trees to Neural Networks

Tree-based methods are powerful but limited by their piecewise-constant predictions. Neural networks learn smooth, continuous functions of arbitrary complexity.

---

# Why Go Beyond Trees?

---

## Tree-based models (L04–L06):

- Excellent for tabular data with many features
- Each row treated **independently** — no built-in notion of order or sequence
- Predictions are piecewise constant: cannot extrapolate beyond training range
- Feature engineering required for temporal patterns (lag columns, rolling means, etc.)

## What neural networks add:

- Learn **smooth, continuous** functions directly
- Flexible architecture: can be adapted for sequences, images, text
- Can learn hierarchical feature representations automatically
- PyTorch enables custom architectures and GPU-accelerated training

Neural networks are not always better than XGBoost on tabular data. They shine when: (1) the data is a sequence (time series, text), (2) raw inputs are images or audio, or (3) you have very large amounts of data ( $n > 100K$ ).

# From Linear Regression to a Single Neuron

**Linear regression:**  $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$

**One neuron** adds a nonlinear **activation**  $\sigma(\cdot)$ :  
 $a = \sigma(\mathbf{w}^\top \mathbf{x} + b)$

| Activation | Formula          | Use               |
|------------|------------------|-------------------|
| ReLU       | $\max(0, z)$     | Hidden layers     |
| Sigmoid    | $1/(1 + e^{-z})$ | Binary / gates    |
| Tanh       | $\tanh(z)$       | Sequence states   |
| Linear     | $z$              | Regression output |

## Why Non-Linearity?

Without  $\sigma(\cdot)$ , stacking layers is equivalent to a single linear model.

Non-linearity enables a network to approximate **any continuous function** to arbitrary accuracy (universal approximation theorem).

## Business intuition

A neuron asks: "Is this pattern present?"  
If yes (large positive  $z$ ), activate. If no, stay off. ReLU makes this literal.

---

## **Feedforward Networks**

Stack layers of neurons. Each layer learns higher-level representations of the input.

---

# Feedforward Network: Layers and Forward Pass

A **feedforward network** (FFN / MLP) has:

- **Input layer:** raw features  $\mathbf{x} \in \mathbb{R}^p$
- **Hidden layers:**  $H$  layers, each applying  $\mathbf{a}^{(\ell)} = \sigma(\mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)})$
- **Output layer:** linear (regression) or softmax (classification)

## Notation:

- $\mathbf{W}^{(\ell)}$ : weight matrix for layer  $\ell$
- $\mathbf{b}^{(\ell)}$ : bias vector for layer  $\ell$
- Width = neurons per layer; Depth = hidden layers

## Practical sizing for forecasting:

- 2–3 hidden layers
- 64–256 neurons per layer
- ReLU activations
- Dropout 0.2–0.5 for regularization

Wider/deeper networks need more data and more regularization.

Bigger is not always better. Start small, add capacity only if the validation error is still too high.

---

## Training: Loss and Backpropagation

How does the network learn? By computing the error, then propagating it backward to update weights.

---



# Training a Network: Loss, Gradients, and SGD

---

**Step 1: Loss** — Regression MSE:

$$\mathcal{L} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

**Step 2: Backpropagation** — work backward to assign each weight its share of the error.

**No calculus required.** `loss.backward()` handles all gradient computation internally.

**Step 3: Update weights (SGD)**

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}}$$

where  $\eta$  = **learning rate**.

## Stochastic Gradient Descent

Use a random **mini-batch** (32–256 samples) per update step instead of the full dataset. One pass through all batches = one **epoch**.

**Use Adam, not plain SGD.** Adam adapts the learning rate per parameter.  
`torch.optim.Adam(lr=1e-3)`

# Preventing Overfitting: Dropout and Early Stopping

---

## Dropout

During training, randomly set a fraction  $p$  of neuron outputs to zero on each forward pass. Neurons cannot rely on specific others being present.

At test time, all neurons are active and outputs are scaled by  $(1 - p)$ .

PyTorch: `nn.Dropout(p=0.3)`

**Early stopping** is essential. Monitor validation loss each epoch. Stop when it fails to improve for  $k$  epochs (patience). Save the model at the best epoch.

## Standard regularization toolkit:

- Dropout ( $p = 0.2-0.5$ )
- Weight decay in Adam (`weight_decay=1e-4`)
- Early stopping (patience = 10–20)
- Batch normalization (deeper nets)

---

# PyTorch Implementation

Dataset, DataLoader, model definition, and training loop.

---

## Custom Dataset

```
from torch.utils.data import Dataset, DataLoader

class ForecastDataset(Dataset):

    def __init__(self, X, y):

        self.X = torch.tensor(X, dtype=torch.float32)

        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self): return len(self.X)

    def __getitem__(self, idx):

        return self.X[idx], self.y[idx]

train_ds = ForecastDataset(X_train, y_train)

train_loader = DataLoader(

    train_ds, batch_size=64, shuffle=True)
```

## Why Dataset and DataLoader?

- Dataset: wraps data, defines how to get one sample
- DataLoader: handles batching, shuffling, and parallel loading
- Separates data handling from model logic

**Time series:** do NOT shuffle. Set `shuffle=False` and use `TimeSeriesSplit` on indices before creating the Dataset.

## Feedforward Regressor

```
import torch.nn as nn

class FFNRegressor(nn.Module):

    def __init__(self, in_dim):

        super().__init__()

        self.net = nn.Sequential(

            nn.Linear(in_dim, 128), nn.ReLU(), nn.Dropout(0.3),

            nn.Linear(128, 64), nn.ReLU(), nn.Dropout(0.3),

            nn.Linear(64, 1))

        def forward(self, x):

            return self.net(x).squeeze(-1)

model = FFNRegressor(in_dim=X_train.shape[1])
```

## Key PyTorch concepts:

- `nn.Module`: base class; override `__init__` and `forward`
- `nn.Sequential`: chains layers in order
- `nn.Linear(in, out)`: FC layer,  $in \times out$  weights + bias
- `nn.ReLU()`: element-wise activation
- `nn.Dropout(p)`: dropout layer

forward defines each network pass. PyTorch computes gradients via autograd.

# PyTorch: The Training Loop

## Training Loop

```
optimizer = Adam(model.parameters(), lr=1e-3)

for epoch in range(100):
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()

        y_hat = model(X_batch)

        loss = nn.MSELoss()(y_hat, y_batch)

        loss.backward()

        optimizer.step()

    model.eval()

    with torch.no_grad():
        val_hat = model(X_val_t)

        val_loss = nn.MSELoss()(val_hat, y_val_t)
```

## Four steps per mini-batch:

1. `zero_grad()`: clear gradients
2. Forward pass: compute  $\hat{y}$
3. `loss.backward()`: compute gradients
4. `optimizer.step()`: update weights

Call `model.eval()` before validation to disable dropout. Use `torch.no_grad()` to skip gradient tracking (saves memory).

---

## Key Takeaways

Neural networks are flexible function approximators. PyTorch provides the building blocks; the training loop is always the same four steps.

---

# Neural Networks vs. Tree Methods

| Method        | Sequences       | Extrapolates | Tuning | Best for                   |
|---------------|-----------------|--------------|--------|----------------------------|
| Random Forest | No (needs lags) | No           | Low    | Tabular, fast baseline     |
| XGBoost       | No (needs lags) | No           | Medium | Tabular, best accuracy     |
| FFN (MLP)     | Partially       | Yes          | Medium | Tabular + smooth functions |
| RNN / LSTM    | Yes             | Yes          | High   | Sequences, time series     |

Lectures 08–09 extend FFN to sequences and images.

## When to use an FFN for forecasting:

- You have engineered lag and rolling features (same as XGBoost)
- You want a smooth, continuous forecast function
- Baseline: XGBoost usually wins on small-to-medium tabular data; FFN may win with larger data or more complex patterns



## Lecture 7: Key Takeaways

---

1. A **neuron** applies a nonlinear **activation** (ReLU, sigmoid, tanh) to a linear combination of inputs. Without non-linearity, stacking layers equals one linear layer.
2. A **feedforward network** stacks neuron layers. Depth adds capacity; width adds breadth.
3. **Training**: compute loss  $\rightarrow$  backpropagate  $\rightarrow$  update with Adam. Use mini-batches for efficiency.
4. **Four steps** per mini-batch: `zero_grad()`  $\rightarrow$  forward  $\rightarrow$  `loss.backward()`  $\rightarrow$  `optimizer.step()`.
5. Prevent overfitting: **dropout**, **weight decay**, **early stopping** on a validation set.
6. For time series, **do not shuffle** the DataLoader. Use **TimeSeriesSplit** for train/val splits.

*Next: CNN Architectures — convolutional filters, pooling, and residual connections.*

# References I

---